



École Supérieure en Informatique
ESI - Sidi Bel Abbès

Lab Report 04

Network Traffic Analysis with Wireshark & tcpdump

Computer Systems and Network Security
Academic Year 2025–2026

Student:
Manel Mostefaoui

Instructor:
Mohammed Salih Bendella

October 29, 2025

1 Objective

The objective of this lab is to learn network traffic analysis using `tcpdump` and Wire-shark. We performed packet capture, analyzed TCP flows, extracted credentials from unencrypted protocols, implemented rotating captures, and identified suspicious network activity. The lab environment consists of Kali Linux 2016 and Metasploitable 2 (IP: 192.168.111.129) connected on a host-only network.

2 Exercise 1: Targeted Capture

2.1 Procedure

We captured network traffic between Kali Linux and Metasploitable for 30 seconds using `tcpdump` with host filtering. The command used was:

```
1 sudo timeout 30 tcpdump -i eth0 host 192.168.111.129 -w /tmp/
target.pcap
```

Before capturing, we generated ICMP traffic using the `ping` command to ensure packets were captured.

2.2 Results

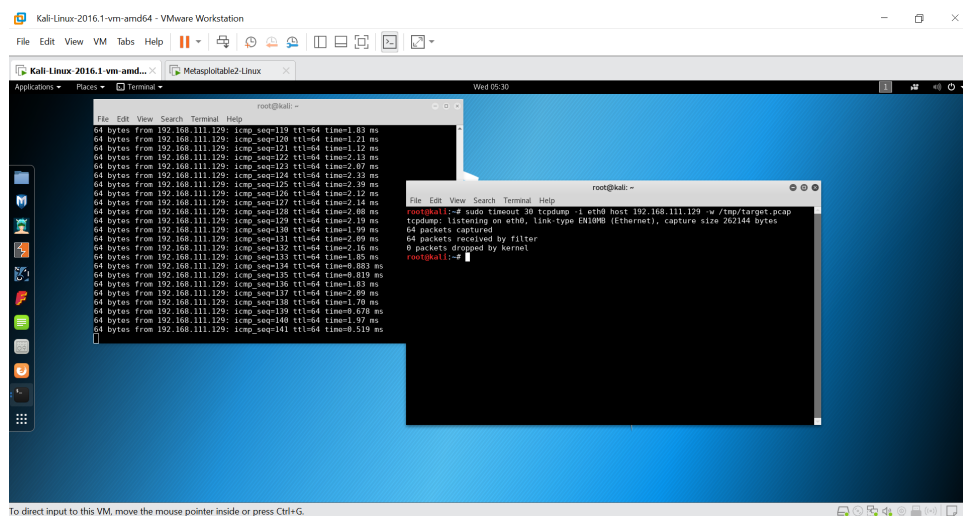


Figure 1: Targeted capture command execution showing 30-second packet capture to `/tmp/target.pcap`

The capture successfully saved packets between the two hosts. The file was verified using `ls -lh` and `tcpdump -r` commands, confirming packets were captured correctly.

3 Exercise 2: Identifying TCP Flows

3.1 Procedure

To analyze TCP flows, we first generated TCP traffic by connecting to Metasploitable via Telnet. Then we opened the capture file in Wireshark and applied a TCP filter to identify the 3-way handshake (SYN, SYN-ACK, ACK).

```
1 telnet 192.168.111.129
2 wireshark /tmp/tcp_traffic.pcap
```

In Wireshark, we used the display filter `tcp` and analyzed the conversation statistics.

3.2 Results

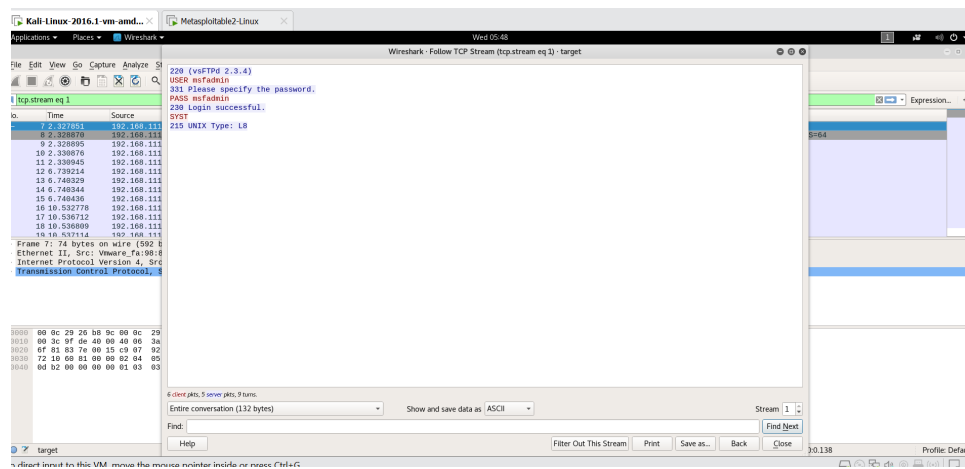


Figure 2: Wireshark showing TCP 3-way handshake with SYN, SYN-ACK, and ACK packets highlighted

```
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
  Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 60
    Identification: 0x0000 (0)
    Flags: 0x02 (Don't Fragment)
    Fragment offset: 0
    Time to live: 64
    Protocol: TCP (6)
    Header checksum: 0xda69 [validation disabled]
    [Header checksum status: Unverified]
    Source: 192.168.111.129
    Destination: 192.168.111.128
    [Source GeoIP: Unknown]
    [Destination GeoIP: Unknown]
  Transmission Control Protocol, Src Port: 21, Dst Port: 33662, Seq: 0, Ack: 1, Len: 0
    Source Port: 21
    Destination Port: 33662
    [Stream index: 1]
    [TCP Segment Len: 0]
    Sequence number: 0 (relative sequence number)
    Acknowledgment number: 1 (relative ack number)
    Header Length: 40 bytes
    Flags: 0x012 (SYN, ACK)
    Window size value: 5792
    [Calculated window size: 5792]
    Checksum: 0xb5d2 [unverified]
```

3.3 Observations

The analysis revealed:

- **Client IP:** 192.168.111.128 (Kali Linux)
- **Server IP:** 192.168.111.129 (Metasploitable)
- **Source Port:** Random high port (ephemeral)
- **Destination Port:** 23 (Telnet)
- **Packets Exchanged:** Multiple packets including handshake and data transfer
- **Flow Duration:** Approximately 2-3 seconds

The TCP stream feature in Wireshark allowed us to view the entire conversation in a single window.

4 Exercise 3: Extracting Credentials in Clear Text

4.1 Procedure

This exercise demonstrated the security risk of unencrypted protocols. We captured Telnet traffic while logging into Metasploitable with credentials (msfadmin:msfadmin).

```
1 sudo tcpdump -i eth0 host 192.168.111.129 -w /tmp/credentials.
   pcap
2 telnet 192.168.111.129
```

After logging in and exiting, we opened the capture in Wireshark and used "Follow TCP Stream" to view the entire conversation.

4.2 Results

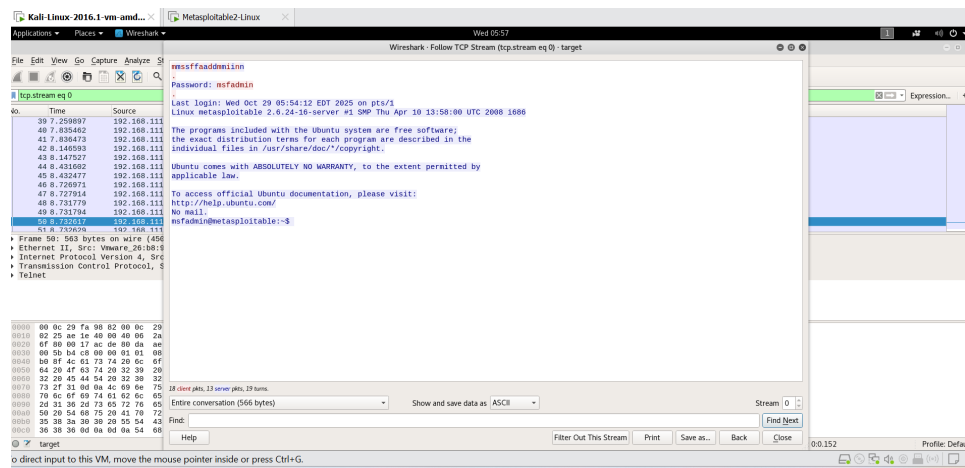


Figure 3: Wireshark TCP Stream showing username and password transmitted in clear text

4.3 Observations

The Telnet protocol transmits all data, including authentication credentials, without encryption. Both the username "msfadmin" and password "msfadmin" were clearly visible

in the TCP stream. This demonstrates why Telnet should never be used for remote access in production environments. SSH should always be preferred as it encrypts all communication.

5 Exercise 4: Capture Rotation and Management

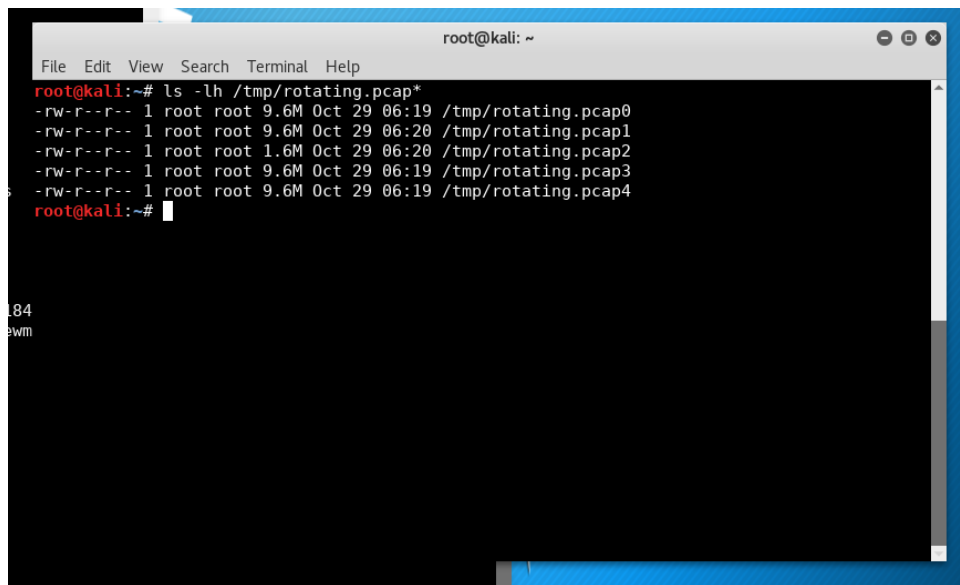
5.1 Procedure

We implemented rotating captures to manage long-term network monitoring. The capture was configured to create 10 MB files and retain only the 5 most recent files.

```
1 sudo tcpdump -i eth0 -C 10 -W 5 -w /tmp/rotating.pcap
```

To generate sufficient traffic, we ran continuous ping and HTTP requests in separate terminals for approximately one minute.

5.2 Results



```
root@kali: ~  
File Edit View Search Terminal Help  
root@kali:~# ls -lh /tmp/rotating.pcap*  
-rw-r--r-- 1 root root 9.6M Oct 29 06:19 /tmp/rotating.pcap0  
-rw-r--r-- 1 root root 9.6M Oct 29 06:20 /tmp/rotating.pcap1  
-rw-r--r-- 1 root root 1.6M Oct 29 06:20 /tmp/rotating.pcap2  
-rw-r--r-- 1 root root 9.6M Oct 29 06:19 /tmp/rotating.pcap3  
-rw-r--r-- 1 root root 9.6M Oct 29 06:19 /tmp/rotating.pcap4  
root@kali:~#
```

Figure 4: Directory listing showing multiple rotating capture files with automatic numbering

5.3 Observations

The rotating capture mechanism successfully created multiple numbered files (rotating.pcap1, rotating.pcap2, etc.). This feature is essential for long-term network monitoring as it prevents disk space exhaustion. When the sixth file would be created, the oldest file is automatically deleted, maintaining only 5 files at any time.